

Segurança da API – resolvendo a Lista 5+6

DIM0547 — Sprint 3 · Vídeo 8 (Parte 2 de Autenticação)

Prof. Fernando · UFRN · 2026.1

Segundo vídeo de autenticação. No anterior implementamos **login + JWT + middleware** (e o `ex01` com bcrypt). Aqui fechamos o que falta para a **Entrega Final: refresh tokens e correções de segurança (OWASP)** — resolvendo a **Lista 5+6** ao longo do caminho.

De onde paramos – e como usar este vídeo

Aula 07 (Vídeo 7) entregou:

- `POST /login` confere a senha com **bcrypt** e devolve um **JWT** (`ex01` + início do `ex02`)
- `AuthMiddleware` valida assinatura, `exp` e `alg` , e injeta o `userID` no context

O que a **Entrega Final (23/06)** ainda exige hoje:

Requisito	Onde resolvemos
Refresh token com rotação	<code>ex04</code>
≥ 2 correções de segurança OWASP	ownership (<code>ex03</code>) + rate limiting (<code>ex04</code>)

Como o vídeo funciona: a **teoria** fica nos slides; a **solução de cada exercício** eu mostro **no IDE**. Quando aparecer o quadro laranja  SOLUÇÃO NO IDE , estou alternando para o editor.

Mapa: Lista 5+6 ↔ Entrega Final

A Lista 5+6 não será publicada como tarefa — resolvemos os exercícios **aqui** e o mesmo código sustenta a entrega do projeto.

Exercício	Conceito	0 que provê para a entrega
ex01	bcrypt + handler testável	base de senhas (Aula 07)
ex02	JWT (access token)	login + rotas protegidas
ex03	ownership (OWASP API3)	1ª correção de segurança
ex04	refresh + rate limiting	rotação + 2ª correção (API4)

Conceitos do repositório: **1** = ex01 ; **2** = ex02 ; **3** = + ex03 ; **4** = + ex04 .

Parte 1 – Tokens de acesso e renovação

Recap: o access token (JWT) da Aula 07

O login devolve um **access token** assinado (HS256), curto (15 min), que o cliente envia em cada request:

```
Authorization: Bearer eyJhbGc...
```

Três cuidados que já vimos:

- **exp curto** — um vazamento tem impacto limitado no tempo
- **alg validado no parse** — defesa contra o ataque `alg:none`
- **erro genérico** no login (`invalid credentials`) — não revela se o email existe

🖥️ SOLUÇÃO NO IDE – ex02 (fechamento)

· `internal/auth/jwt.go` → `GenerateAccessToken` / `ValidateAccessToken` · `handler/auth.go` → `Login`
emitindo o token

0 dilema do tempo de vida

exp curto (minutos)	exp longo (dias)
└ vazamento dura pouco	└ vazamento dura muito
└ usuário relogga toda hora (péssima UX)	└ usuário fica logado (boa UX, péssima segurança)

Segurança e boa experiência puxam para lados opostos. Um único token não resolve.

A saída é **separar responsabilidades**: um token para *acessar* (curto, usado o tempo todo) e outro para *renovar* (longo, usado raramente). Cada um otimiza um objetivo.

Dois tokens, dois papéis

	Access token	Refresh token
Função	Provar identidade em cada request	Obter um novo access token
Tempo de vida	Curto — 15 min	Longo — 7 dias
Usado	Em toda requisição	Só quando o access expira
É stateless?	Sim — JWT autocontido	Não — guardado (em <i>hash</i>) no banco

O token que viaja muito (access) dura pouco. O que dura muito (refresh) quase não viaja — e, por estar no banco, **pode ser revogado**.

Por que o refresh fica no banco

```
CREATE TABLE refresh_tokens (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token_hash  TEXT NOT NULL,           -- hash, nunca o token original  
  expires_at  TIMESTAMPTZ NOT NULL,  
  revoked_at  TIMESTAMPTZ              -- NULL = ativo  
);
```

Modelagem 1:N da Sprint 2 — um usuário tem muitos refresh tokens (um por dispositivo). `revoked_at NULL` é o estado "ativo"; preencher a data é revogar. Guardamos o **hash** (SHA-256): vazou o banco, os tokens não funcionam.

0 fluxo: rotação e detecção de reuso

```
login          → access + refresh-1   (refresh-1 salvo como hash)
/refresh (rt-1) → revoga rt-1, emite access + refresh-2   (ROTAÇÃO)
/refresh (rt-1) → rt-1 já revogado → REUSO detectado → 401
                  + revoga TODOS os refresh do usuário
```

- **Rotação**: cada `/refresh` invalida o token usado e emite um novo. O cliente sempre tem o último.
- **Detecção de reuso**: se um refresh **já revogado** reaparece, é sinal de roubo → revoga toda a família e força novo login.

A ordem importa: a checagem de **reuso vem antes da de expiração**. Um token revogado que volta não é "expirado", é suspeito.

Logout: o que o refresh resolve

Token	Após o logout
Access token	Ainda válido — mas expira em minutos
Refresh token	Revogado imediatamente — não renova mais

O `/logout` revoga o refresh e devolve **204 sempre** — exista ou não o token (idempotente). Diferenciar vazaria se aquele token chegou a existir.

SOLUÇÃO NO IDE – ex04 (refresh tokens)

- `internal/auth/token.go` → `GenerateRefreshToken` (crypto/rand) / `HashRefreshToken` (SHA-256) .
- `handler/auth.go` → `Login` (emite o par), `Refresh` (rotação + reuso), `Logout`

Parte 2 – Autorização e Segurança (OWASP)

Autenticação ≠ Autorização

	Pergunta	Quem resolve
Autenticação	<i>Quem é você?</i>	login + JWT (Aula 07)
Autorização	<i>Você pode fazer isto?</i>	regra de negócio no handler

O JWT prova **quem** é o usuário. Ele **não** decide o que esse usuário pode acessar — isso é com a gente, em cada rota.

401 Unauthorized → não sei quem você é (falta/inválido o token)
403 Forbidden → sei quem é, mas não pode (sem permissão)
404 Not Found → o recurso não existe... ou você não pode saber que existe

A distinção 401/403/404 é o vocabulário de toda esta parte. Guarde o **404 com segundo sentido** — voltamos a ele no BOLA.

OWASP API Security Top 10

A **OWASP** mantém uma lista dos riscos mais comuns. APIs têm a **sua própria** lista (separada da web tradicional), porque os ataques são diferentes: não há tela, e o cliente fala direto com os endpoints.

Hoje abordaremos as duas mais frequentes:

Risco	Nome	Onde, no projeto
API3:2023	<i>Broken Object Level Authorization (BOLA)</i>	acesso às notas (ex03)
API4:2023	<i>Unrestricted Resource Consumption</i>	força bruta no login (ex04)

BOLA é, há anos, o **risco nº 1** em APIs. É também o mais simples de explorar — e, por isso, o mais cobrado em revisão.

API3 – BOLA, explicado

Object Level Authorization = verificar, a cada acesso, se *aquele objeto específico* pertence a quem está pedindo.

O ataque (também chamado **IDOR**):

```
Ana está logada e acessa a própria nota:  
GET /notes/7 (Authorization: Bearer <token da Ana>)
```

```
Ana troca o id na URL:  
GET /notes/8 ← a nota 8 é de João
```

Se a API devolver a nota 8, **quebrou** o controle de autorização por objeto. O token da Ana é válido (autenticação OK), mas ela **não é dona** da nota 8 (autorização falhou).

A falha não está no token — está em **esquecer de checar o dono**. Autenticar não é autorizar.

API3 – a defesa: 404, não 403

A regra de ouro da autorização: recurso de outro usuário → 404, nunca 403.

403 → "existe, mas você não pode" ← confirma que a nota 8 existe
404 → "não encontrado" ← a existência fica indistinguível

🖥️ **SOLUÇÃO NO IDE – ex03** (ownership)

· `handler/notes.go` → `Get` / `Update` / `Delete` traduzindo `ErrNoteNotFound` em 404

API4 – limitação de taxa

APIs gastam recursos a cada request: CPU, banco, e-mail, SMS. Sem limite, um cliente abusivo consegue:

- **Força bruta** de senha no `/login` — milhares de tentativas por segundo
- **Negação de serviço** (DoS) — afogar o servidor de requisições
- Estouro de custo em serviços pagos (envio de SMS, etc.)

A defesa é **limitar a taxa** (rate limiting) por origem (IP):

```
até 5 req/s por IP, com folga (burst) de 10  
acima disso → 429 Too Many Requests (+ header Retry-After)
```

O `/login` é o alvo clássico de força bruta — por isso o rate limit entra exatamente nele. O algoritmo é o **token bucket**: cada IP tem um "balde" que reabastece a 5 fichas/s.

API4 – rate limiting no login

O middleware fica **na frente** do handler de login: lê o IP de origem, consulta o limiter daquele IP e, se estourou, corta a requisição com **429** antes de tocar no banco.

- Um `rate.Limiter` por IP, guardados num `map` protegido por mutex
- `429` com `Retry-After: 1` e corpo JSON `{"error":"rate limit exceeded"}`

SOLUÇÃO NO IDE – ex04 (rate limiting)

- `middleware/rate_limit.go` → `Middleware` (`SplitHostPort` → `getLimiter` → `Allow` → `429`) · `cmd/api/main.go`
→ onde o limiter é plugado só no `/login`

As correções de segurança, em um quadro

Correção	OWASP / risco	Onde
Ownership por objeto (404, não 403)	API3:2023 BOLA	ex03
Rate limiting no login	API4:2023	ex04
Mensagem genérica "invalid credentials"	enumeração de usuário	ex02
<code>alg</code> validado no parse	ataque <code>alg:none</code>	ex02
Senha em bcrypt , refresh em SHA-256	vazamento de banco	ex01 / ex04
<code>JWT_SECRET</code> em <i>env var</i>	segredo no repositório	config

Para a entrega, **duas** correções já bastam.

Referências

Refresh tokens e sessão

- [OWASP — Session Management Cheat Sheet](#)
- `golang-jwt/jwt` · `golang.org/x/time/rate`

OWASP API Security

- [OWASP API Security Top 10 \(2023\)](#) — visão geral
- [API3:2023 — Broken Object Level Authorization](#)
- [API4:2023 — Unrestricted Resource Consumption](#)